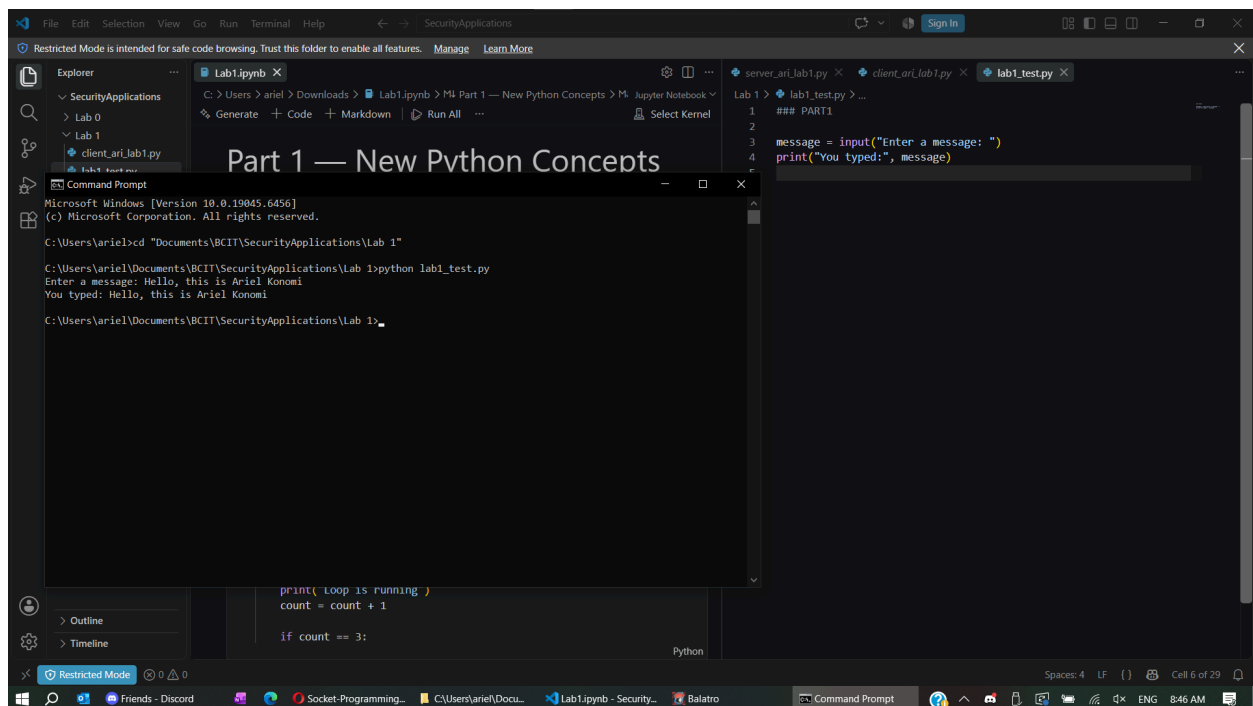
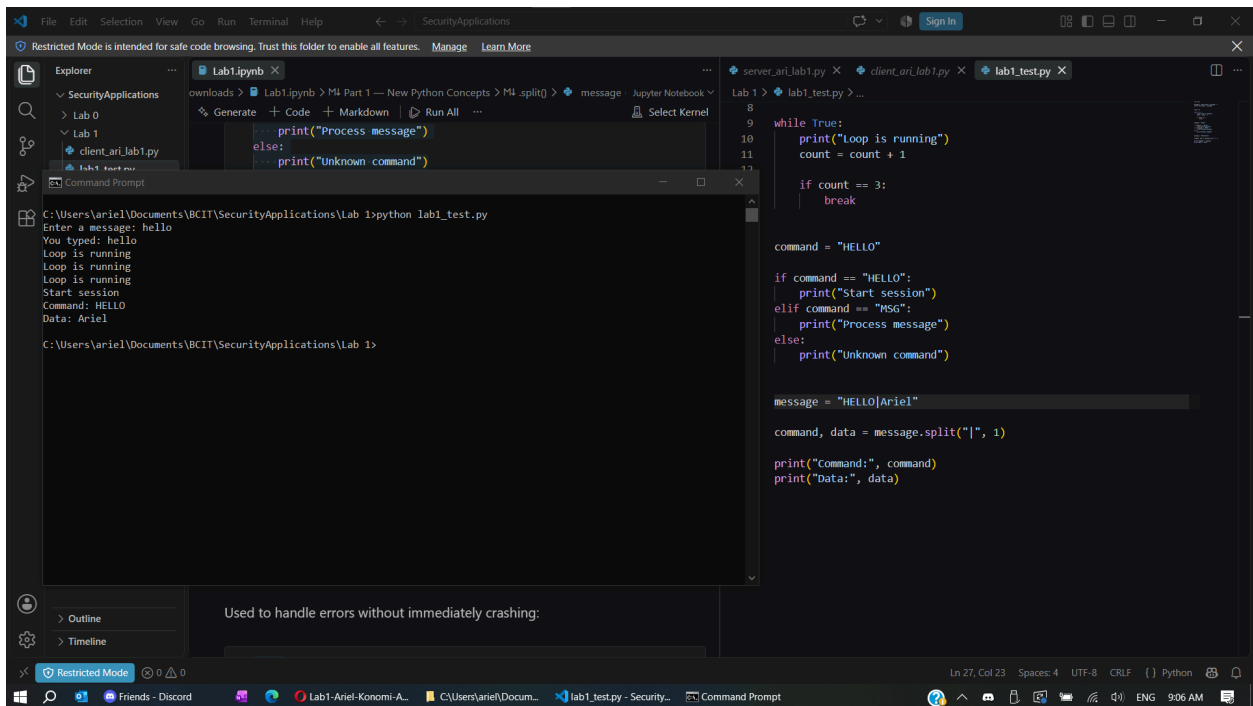
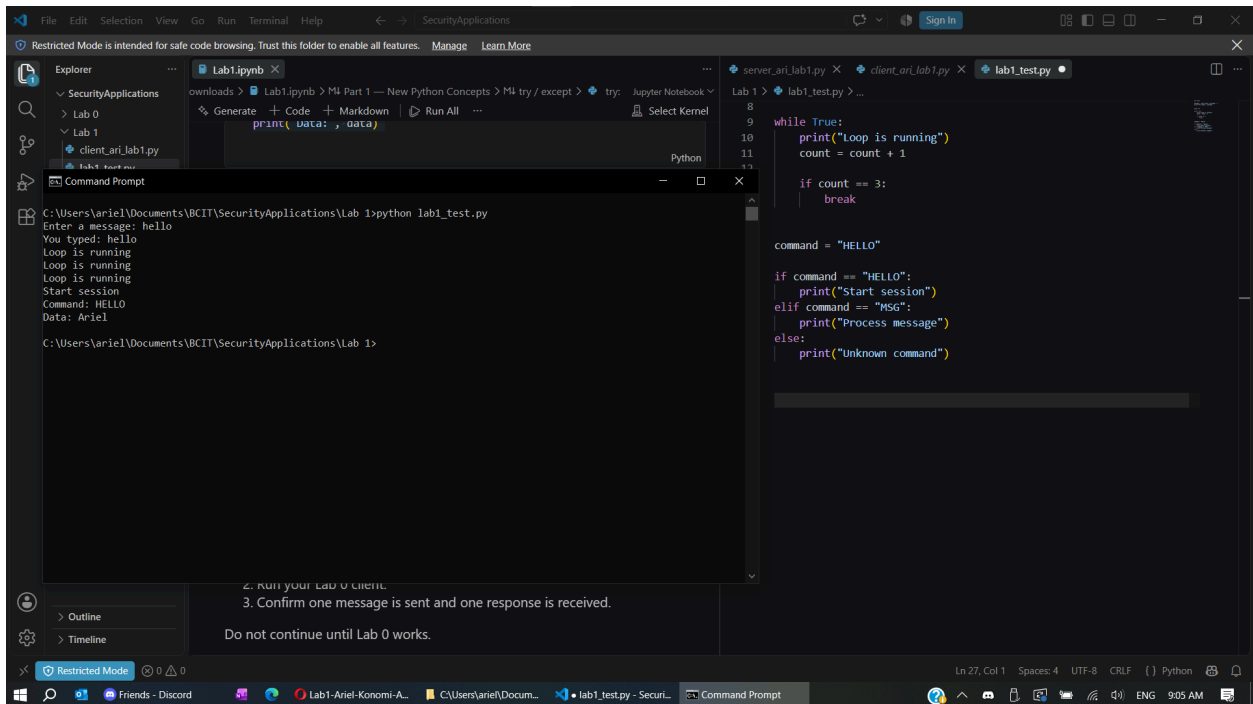


Lab 1: Developing a stateful Client-Server Chat Application in Python

Part 1





```
command, data = message.split("|", 1)

print()
print()

command = "HELLO"

if command == "HELLO":
    print("Start session")
elif command == "MSG":
    print("Process message")
else:
    print("Unknown command")

print()
print()

message = "HELLO|Ariel"

command, data = message.split("|", 1)

print("Command:", command)
print("Data:", data)

print()
print()

try:
    number = int("abc")
except ValueError:
    print("Invalid value")
```

Before adding anything new.

1. Run your Lab 0 server.
2. Run your Lab 0 client.

Part 2

Confirmed that the server and client are in working condition from lab 0

```
try:
    number = int("abc")
except ValueError:
```

Server received:Hello Server. This is for Part 8 of Lab0 - Ariel Konomi

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1>

The key difference in Lab 1 is that the client and server should n one message.

Part 3

This part was a bit tough to be honest as I tried writing my own script from scratch. Here is a screenshot of it being half correct:

The screenshot shows a Jupyter Notebook titled "Lab1.ipynb" with a code cell containing the following Python code:

```
# Part 3 - Make the Connection Persistent
client_socket.send(message.encode())
response = client_socket.recv(1024)
response = response.decode()
```

The Command Prompt shows the output of the client script, which includes the message "Hello Server. This is for Part 3 of Lab1 - Ariel Konomi". The server script is also shown, which is waiting for a connection. The output of the server script shows that it received the message and responded with "Hello Server. This is for Part 3 of Lab1 - Ariel Konomi".

Below is the fixed client and server:

The screenshot shows a Jupyter Notebook titled "Lab1.ipynb" with a code cell containing the following Python code:

```
# Part 3 - Make the Connection Persistent
client_socket.send(message.encode())
response = client_socket.recv(1024)
response = response.decode()
```

The Command Prompt shows the output of the client script, which includes the message "Hello Server. This is for Part 3 of Lab1 - Ariel Konomi". The server script is also shown, which is waiting for a connection. The output of the server script shows that it received the message and responded with "Hello Server. This is for Part 3 of Lab1 - Ariel Konomi".

Part 4

All client messages must follow:

```
``text
COMMAND|DATA
``
```

Use these commands:

```
``text
HELLO|username
MSG|text
EXIT|
``
```

Examples:

```
``text
HELLO|Maryam
MSG|Hello everyone
MSG|How are you?
EXIT|
``
```

The server responds with:

```
``text
OK|
``
```

or

```
``text
ERROR|reason
``
```

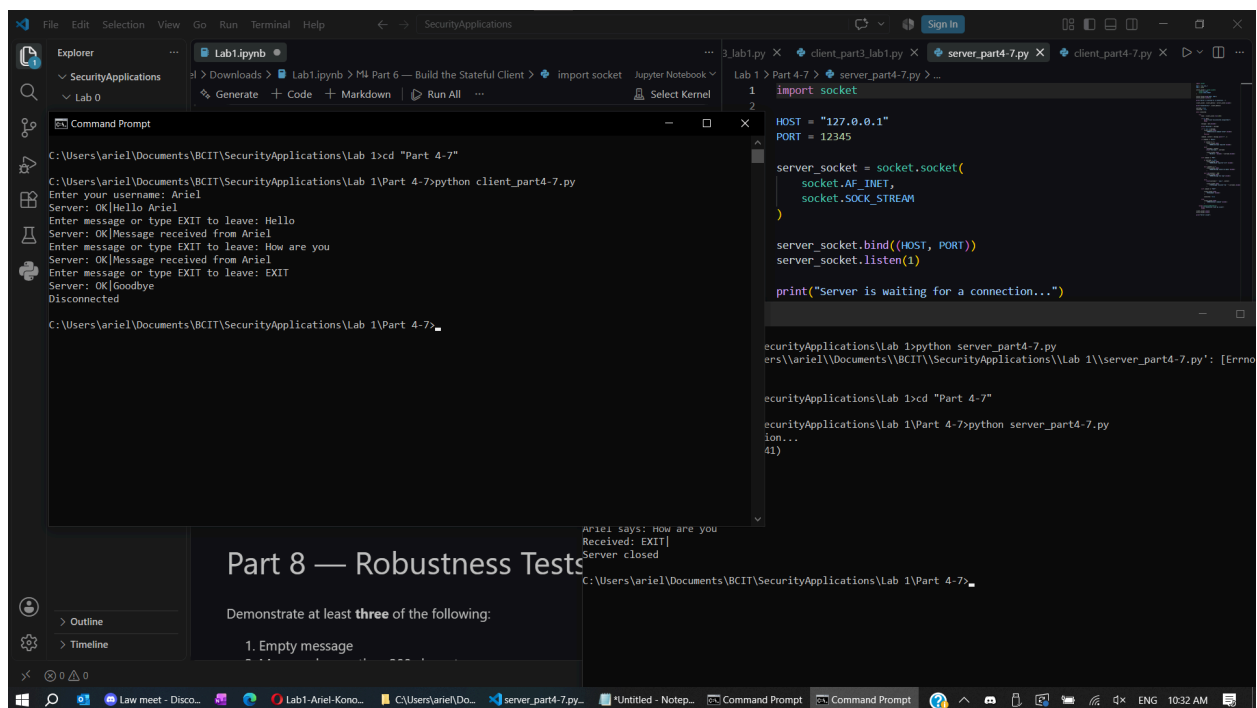
Part 5

The server script has been updated as per the instructions

Part 6

The client script has also been updated as per the instructions

Part 7



The screenshot displays a JupyterLab environment with a dark theme. The Explorer panel on the left shows the file structure, including a folder named 'SecurityApplications' and a file 'Lab1.ipynb'. The main editor area shows a Python script for a server. The script defines a host and port, creates a socket, binds it, and listens for connections. The terminal window shows the execution of the script, including the command to run the server and the output of the script.

```
HOST = "127.0.0.1"
PORT = 12345

server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

server_socket.bind((HOST, PORT))
server_socket.listen(1)

print("Server is waiting for a connection...")
```

The terminal window shows the following output:

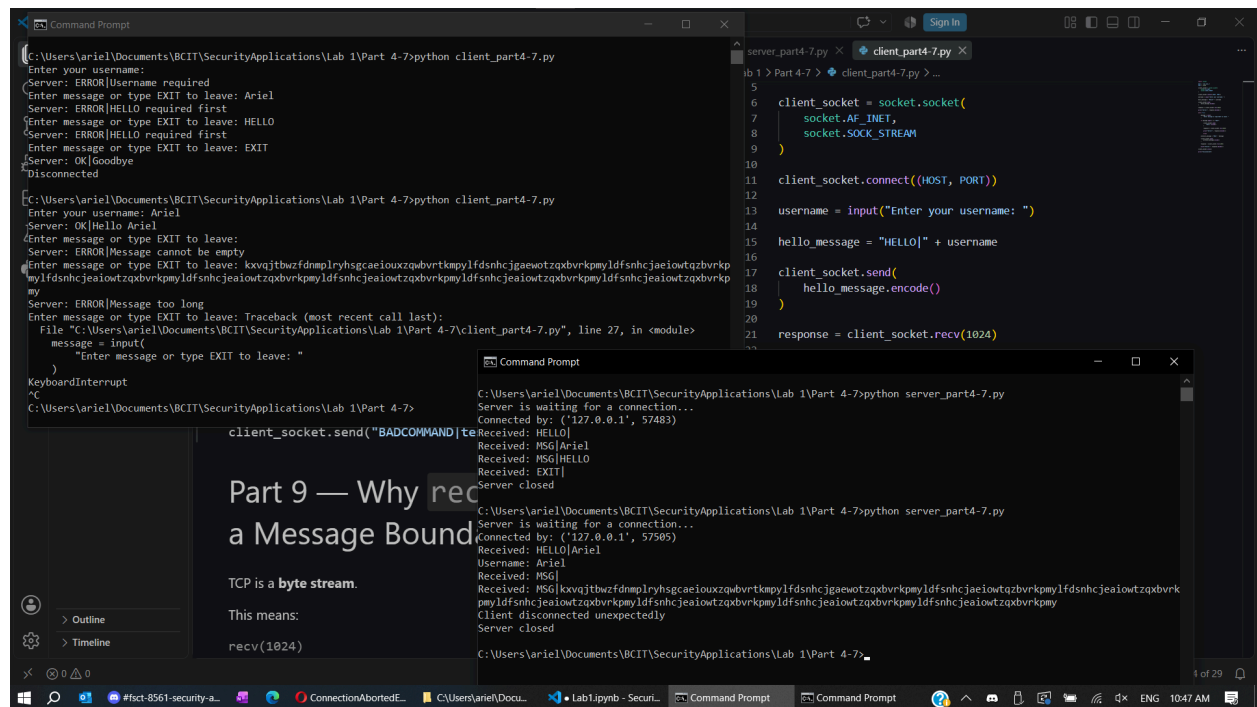
```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1>cd "Part 4-7"
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-7>python client_part4-7.py
Enter your username: Ariel
Server: OK[Hello Ariel
Enter message or type EXIT to leave: Hello
Server: OK[Message received from Ariel
Enter message or type EXIT to leave: How are you
Server: OK[Message received from Ariel
Enter message or type EXIT to leave: EXIT
Server: OK[Goodbye
Disconnected
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-7>
```

Below the terminal window, there is a section titled 'Part 8 — Robustness Tests' with the following text:

Demonstrate at least **three** of the following:

1. Empty message

All of the robustness tests are within this one screenshot:



Part 9

TCP is a ****byte stream****.

This means:

```
``python
recv(1024)
``
```

does **not** guarantee that one complete application message arrives in one call.

A long message may be split across multiple `recv()` calls, or multiple small messages may arrive together.

For this lab, the simple protocol is acceptable for learning purposes, but a production system needs proper ****message framing****.

Part 10

Security Reflection

Answer:

1. What happens if the server crashes while the client is connected?
 - a. The client will be unable to connect and will get an unexpected error as the server closes its sockets
2. How could the server be extended to handle multiple clients?
 - a. The server could be extended to handle multiple clients by using threading to connect to each incoming accept command
3. Why can `recv(1024)` split messages unexpectedly?
 - a. `recv(1024)` can split messages unexpectedly due to MTU size limits. It only fetches up to 1024 bytes.
4. Why does TCP reliability not mean the application is secure?
 - a. TCP is reliable in ensuring that bytes arrive in order. It does not ensure security of any attacks like packet sniffing or man in the middle, nor does it ensure authenticity
5. What new attack surfaces are created by adding usernames and protocol commands?
 - a. The identity of a user can be spoofed. Clients can also try to send MSG commands before a HELLO command which might break the state of the machine
6. How could a malicious client abuse the protocol?
 - a. A malicious client can try impersonating an admin or can repeatedly cause connection errors to tire out resources
7. How would you add authentication?
 - a. When first trying to connect to the server, I would ask the client to enter their username and password. This would be a step of authentication
8. How would you add encryption?
 - a. I would add encryption by using TLS (transport layer security)
9. What validation is performed in your implementation?
 - a. A message cannot be longer than 200 characters in length
 - b. It does not allow for further communication with the server if the username OR message is empty

10. What additional mitigations would be needed for a real deployment?
 - a. TLS encryption
 - b. Implementing connection timeouts if there are unresponsive client connections

Part 11

```
File Edit Selection View Go Run Terminal Help
SecurityApplications
Lab1.ipynb
server_part11.py client_part11.py
1 > Part 11 > client_part11.py > ...
2 #####socket.io
3
4 import socketio
5
6 sio = socketio.Client()
7
8
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 11>python client_part11.py
Connection established by Ariel Konomi
Type a message (or EXIT to quit): Hello
Type a message (or EXIT to quit): This is a test
Type a message (or EXIT to quit): Lab 1 from Ariel Konomi
Type a message (or EXIT to quit): A01375274
Type a message (or EXIT to quit): EXIT
Ariel disconnected from server
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 11>

Server: OK|Goodbye
Disconnected

Part 8 — Robustness Test
Demonstrate at least three of the following:
1. Empty message

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 11>python server_part11.py
===== Running on http://0.0.0.0:8080 =====
(Press CTRL+C to quit)
Client connected: pMLQpHsfJF2wq33AAAB
Received from pMLQpHsfJF2wq33AAAB: Hello
Received from pMLQpHsfJF2wq33AAAB: This is a test
Received from pMLQpHsfJF2wq33AAAB: Lab 1 from Ariel Konomi
Received from pMLQpHsfJF2wq33AAAB: A01375274
Client disconnected: pMLQpHsfJF2wq33AAAB
```

Discussion

Which low-level operations from this lab are hidden by Socket.IO?

- a. `recv()` and `send()` are removed from the script
- b. `accept()` is removed from the script
- c. `socket()` is removed from the script

Challenge

```
server_challenge.py
27 response = client_socket.recv(10
28 print("Server:", response.decode
29
30 # Now we start the background th
31 listener = threading.Thread(targ
32 listener.daemon = True
33 listener.start()
34
35 # Main loop is now just for typi
36 while True:
37     message = input()
```

Command Prompt - python client1_challenge.py

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python client1_challenge.py
Enter your username: Ariel
Server: OK|Hello Ariel
Hello
OK|Message sent
Konomi says: Hi Ariel
Konomi says: This is a very interesting challenge
This is the challenge for lab 1. Ariel Konomi - A01375274
OK|Message sent
```

Command Prompt - python client2_challenge.py

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python client2_challenge.py
Enter your username: Konomi
Server: OK|Hello Konomi
Ariel says: Hello
Hi Ariel
OK|Message sent
This is a very interesting challenge
OK|Message sent
Ariel says: This is the challenge for lab 1. Ariel Konomi - A01375274
```

```
server_challenge.py
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python server_challenge.py
Server is waiting for connection(s)...
User connected: Ariel
User connected: Konomi
Connection reset by client
Connection reset by client
A client disconnected
A client disconnected
Connection reset by client
Connection reset by client
A client disconnected
A client disconnected
```

Command Prompt

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python client2_challenge.py
Enter your username: Konomi
Server: OK|Hello Konomi
Enter message or type EXIT to leave: Hello Ariel
Enter message or type EXIT to leave:
OK|Message sent
Ariel says: Hello Konomi
EXIT
Disconnected
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>
```

Command Prompt - python server_challenge.py

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python server_challenge.py
Server is waiting for connection(s)...
User connected: Ariel
User connected: Konomi
Connection reset by client
Connection reset by client
A client disconnected
A client disconnected
Connection reset by client
Connection reset by client
A client disconnected
A client disconnected
```

Challenge Reflection

Answer briefly:

1. Why could the original Lab 1 server handle only one client at a time?
 - a. The original server could only handle one client at a time as threading was not used. The moment a client connected, the client would not try to listen for other connections from other clients
2. What problem does threading solve?
 - a. Threading allows the server to continue listening for other clients, not just one as it creates other threads for the connected clients
3. What happens to the other clients when one client disconnects?
 - a. The other clients would remain unaffected, just like it would in real life servers and communication apps
4. What additional security risks arise when multiple clients share the same server?
 - a. Some security risks would be chat spamming as well as manipulation from one of the clients due to the server broadcasting messages to clients and not fully checking the contents
5. What could happen if two clients try to use the same username?
 - a. Due to there being no validation, in my case, both clients can have the same name

```
File Edit Selection View Go Run Terminal Help SecurityApplications Sign In
server_challenge.py client1_challenge.py client2_challenge.py
Command Prompt - python client1_challenge.py
Server: OK|Hello Ariel
Hello
OK|Message sent
Konomi says: Hi Ariel
Konomi says: This is a very interesting challenge
This is the challenge for lab 1. Ariel Konomi - A01375274
OK|Message sent
EXIT
Disconnected
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python client
Traceback (most recent call last):
  File "C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge\client
    client_socket.connect((HOST, PORT))
ConnectionRefusedError: [WinError 10061] No connection could be made because the
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python client
Enter your username: Ariel
Server: OK|Hello Ariel
hi
OK|Message sent
Ariel says: hi
KeyboardInterrupt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python server_challenge.py
Server is waiting for connection(s)...
User connected: Ariel
User connected: Ariel
Command Prompt - python server_challenge.py
fd, addr = self._accept()
KeyboardInterrupt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python client2_challenge.py
Enter your username: Traceback (most recent call last):
  File "C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge\client2_challenge.py", line 22, in <module>
    username = input("Enter your username: ")
KeyboardInterrupt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>
Ariel says: Hello
Hi Ariel
OK|Message sent
This is a very interesting challenge
OK|Message sent
Ariel says: This is the challenge for lab 1. Ariel Konomi - A01375274
EXIT
Disconnected
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>python client2_challenge.py
Enter your username: Traceback (most recent call last):
  File "C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge\client2_challenge.py", line 22, in <module>
    username = input("Enter your username: ")
KeyboardInterrupt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Challenge>
Ariel says: hi
hi
OK|Message sent
```

6. How would you prevent one user from impersonating another user?
 - a. A username and password would probably be a good way to check and prevent impersonation

Analysis

In this lab, I used raw TCP protocol. Normally TCP only guarantees packet delivery. This means that there is no guarantee of any security while the packet delivery is happening, so usernames and messages are sent via plaintext. Like I wrote in the reflection questions for the previous parts, this would leave a chance open for man in the middle attacks or packet sniffing. While doing this lab, I went from only one connection to the server, to being able to create a multiple connection one. Threading was used in this scenario, which allows the server to create background connections with the clients, and leaving the main thread open to always check for new clients trying to connect. There could be a few scenarios where the server can be threatened if the script is left as it is in the lab. A client can easily impersonate another client or user, as the server does no sort of authentication. They can easily open multiple connections using threading which would exhaust the server's resources, making it difficult for actual clients to be able to use the server for important means. To secure the server and clients, TLS would need to be used. This would encrypt the data while in transit. Regarding a user's or client's identity, it would be best to add some sort of authentication method, in this case a username and password. Connection timeouts would be important as well, to ensure that no client connection can be hijacked and used by someone malicious. Being able to place a threading cap would also be important. This would ensure that only a certain amount of connections can be made, lowering the risk of or chance of a DoS attack from a malicious user or client.

Helpful guides

<https://realpython.com/intro-to-python-threading/>

<https://docs.python.org/3/library/threading.html>

<https://medium.com/@speaktoharisudhan/threading-in-python-dba88fe8a4d8>

<https://discuss.python.org/t/simple-multi-client-chat-using-threads/73043>